Общие требования

Технические и архитектурные требования

Приложение должно быть реализовано с использованием следующих технологий:

- Язык программирования: *Kotlin*;
- Интерфейс пользователя: *Jetpack Compose* и *Material Design 3*;
- Архитектурные подходы:
    a) Разделение обязанностей между интерфейсом пользователя и бизнес-логикой должно быть реализовано с помощью паттерна *MVVM (Model-View-ViewModel)*;
    b) Взаимодействие между *View* и *ViewModel* должно быть строго циклическим и однонаправленным (концепция *UDF*);
- Работа с реляционной БД: *Room* и *Kotlin Flow*;
- Вспомогательные хранилища: *Realm*, *Jetpack DataStore* и *kotlinx.serialization* для сериализациия *JSON*-файлов;
- Сетевые запросы: *Retrofit* и *OkHttp*;
- Асинхронность: *Kotlin Coroutines* и *Kotlin Flow*;
- Работа с камерой: *CameraX* и *Google ML Kit* для распознования штрихкодов и текстов на изображении;
- Сборщик проекта: *Gradle*.

В заданиях, где данные получаются через *REST API*, нет необходимости реализовывать собственный сервис. Допускается создать *mock*-сервер в *Postman* (или другим аналогичным способом) или использовать любой открытый *API*-сервис, подходящий по тематике задания.

Примеры приложений

Now in Android App – пример приложения от *Google*, реализованное с использованием *Kotlin* и *Jetpack Compose*. Оно следует лучшим практикам проектирования и разработки *Android*-приложений и призвано помочь разработчикам оставаться в курсе последних новостей в мире *Android*-разработки.

Требования к отчету по лабораторным работам

Отчет должен обязательно включать следующую информацию:

- Ссылку на открытый *GitHub* репозиторий;
    - Репозиторий должен содержать **README.md файл с описанием всего функционала приложения и скриншотами**. После выполнения всех работ по описанию проекта должно быть наглядно понятно какие функции выполняет приложение и как их использовать.
- Снимки экранов и функционала, описанного в задании;

- Пример кода работы с хранилищами данных (БД, *JSON*-файлы, *Jetpack DataStore* и т.д.);
- Пример кода описания пользовательского интерфейса;
- Пример кода реализации шаблона *MVVM*;
- Пример кода получения данных по сети (*REST, Retrofit, OkHttp, Kotlin Coroutines* и *Kotlin Flow*);
- Пример кода для работы с камерой и распознаванием штрихкодов (если указано в индивидуальном задании);
- Пример кода для работы с *LLM;*
- Пример кода для работы с голосовым интерфейсом;
- Пример кода реализации архитектурных особенностей (если указано в индивидуальном задании).

Демонстрация готового приложения

Приложение должно быть запущено на реальном устройстве. При отсутствии реального устройства, по согласованию с преподавателем, возможен запуск в эмуляторе.

Лабораторная работа №1

Проектирование и реализация пользовательского интерфейса средствами фреймворка *Jetpack Compose* и *Material Design 3*

Цель работы: освоить декларативный подход к построению пользовательских интерфейсов на платформе *Android* с использованием фреймворка *Jetpack Compose*. Изучить базовые функции, механизмы компоновки, управления состоянием и навигации. Овладеть инструментами дизайн-системы *Material Design 3* для создания современных, адаптивных и стилизованных интерфейсов. Научиться применять темизацию, адаптивную верстку и анимации.

Задание

Реализовать часть проекта, помеченную зеленым цветом, в соответствии со своим вариантом.

Контрольные вопросы

1. В чем заключается суть декларативного подхода *Jetpack Compose* и его принципиальное отличие от *XML*-разметки?
2. Как устроен жизненный цикл *Composable*-функций?
3. В чем разница между контейнерами *Column*, *Row, Box*?
4. В чем разница между *remember*, *mutableStateOf* и *rememberSaveable*? Как они помогают сохранять состояние при рекомпозиции и повороте экрана?
5. Как связать бизнес-логику (*ViewModel*) с интерфейсом (*Jetpack Compose*) в архитектуре *MVVM*?
6. Как устроена навигация в *Jetpack Compose*? Опишите роли *NavController* и *NavHost*.
7. Какую роль выполняет компонент *Scaffold* в *Material Design 3*?
8. Как устроена работа с большими списками в *Jetpack Compose*? В чем отличие *LazyColumn* от *RecyclerView* и как оптимизировать производительность списков?
9. Как в *Jetpack Compose* реализуется адаптивная верстка?
10. Какие инструменты существуют в *Jetpack Compose* для реализации анимаций?

Лабораторная работа №2

Архитектурное проектирование приложений под *Android* на основе паттерна *Model-View-ViewModel* и принципов внедрения зависимостей

Цель работы: изучить шаблоны проектирования мобильного программного обеспечения. Научиться разделять бизнес-логику, данные и интерфейс приложения с использованием архитектуры *MVVM (Model-View-ViewModel)*. Освоить принципы слабой связанности компонентов посредством механизмов внедрения зависимостей *(Dependency Injection)*.

Задание

Реализовать часть проекта, помеченную лиловым цветом, в соответствии со своим вариантом.

Контрольные вопросы

1. В чем заключается сущность архитектурного шаблона *Model-View-ViewModel*? Опишите назначение каждого компонента.
2. В чем заключаются фундаментальные различия между паттернами *MVC*, *MVP* и *MVVM* в мобильной разработке?
3. Зачем нужен класс *ViewModel* при управлении жизненным циклом приложения?
4. Как сопрягаются методология *Clean Architecture* и шаблон *MVVM*?
5. В чем заключаются ключевые различия между *LiveData*, *StateFlow* и *SharedFlow*?
6. Как устроена логическая модель однонаправленного потока данных *(UDF)* в связке *ViewModel* и *Jetpack Compose*?
7. Для чего используются изолированные классы и интерфейсы при моделировании состояний экрана (*Loading, Success, Error*)?
8. Как *ViewModel* управляет жизненным циклом асинхронных операций *(Kotlin Coroutines)?*
9. Как принцип инверсии зависимостей и паттерн *Dependency Injection* решают проблему жесткой связанности компонентов в *Android*-приложении?
10. В чем заключаются архитектурные различия между компилируемым *DI*-фреймворком *Dagger Hilt* и динамическим сервис-локатором *Koin*?

Лабораторная работа №3

Механизмы сериализации, локального и долговременного хранения данных в ОС *Android*

Цель работы: изучить подходы к организации локального и долговременного хранения данных в ОС *Android*. Освоить механизмы сериализации и управления базами данных.

Задание

Реализовать часть проекта, помеченную желтым цветом, в соответствии со своим вариантом.

Контрольные вопросы

1. Какие типы и объемы данных допустимо хранить в *DataStore*?
2. Как библиотека *kotlinx.serialization* обеспечивает типобезопасность и поддержку *null*-значений при десериализации *JSON*?
3. В чем заключаются различия между реляционным подходом на базе *SQLite* и объектно-ориентированным подходом СУБД *Realm*?
4. Какую роль выполняют компоненты *RoomDatabase* и *Dao* в архитектуре слоя данных?
5. Как описываются связи между таблицами (один-к-одному, один-ко-многим) в *Room*?
6. Зачем нужна миграция схемы данных в *Room*?
7. Как организовать сквозную передачу отфильтрованных и отсортированных данных из локальной базы напрямую в интерфейс с помощью реактивных потоков *Flow*?
8. Как безопасно выполнять запросы к базам данных из разных потоков?
9. Как работает концепция ленивой загрузки *(Lazy Loading)* объектов из базы данных?
10. Какие подходы и библиотеки используются для полного шифрования файлов баз данных?

Лабораторная работа №4

Организация сетевого взаимодействия на основе архитектурного стиля *REST*.
Реактивное программирование асинхронных потоков данных

Цель работы: овладеть навыками асинхронного взаимодействия с веб-службами посредством архитектурного стиля *REST* с использованием библиотеки *Retrofit*. Освоить реактивный подход для обработки, фильтрации и трансформации асинхронных потоков данных с помощью *Kotlin Flow*.

Задание

Реализовать часть проекта, помеченную голубым цветом, в соответствии со своим вариантом.

Контрольные вопросы

1. Опишите последовательность шагов для выполнения базового сетевого запроса с использованием *Retrofit*.
2. Что такое *Kotlin Flow*? Опишите его фундаментальные компоненты.
3. С помощью какого метода *Retrofit* можно получить данные асинхронно?
4. Как связать цепочку операторов *Flow* со слоем данных и трансформировать сырые бинарные данные в готовые объекты?
5. Что такое *Job* и *CoroutineScope*?
6. Для чего используются операторы *.flowOn* и *.catch*?
7. Какую роль выполняют фабрики конвертеров *(Converter.Factory)* в *Retrofit*?
8. Что такое сетевые перехватчики *(Interceptors)*?
9. Что такое реактивный поток данных в *Kotlin* и из каких фундаментальных компонентов он состоит?
10. В чем разница между реактивным подходом (потоки данных *Flow*) и последовательным асинхронным программированием *(suspend/async)* при организации сетевых запросов?

Лабораторная работа №5

Интеграция больших языковых моделей в *Android*-приложение

Цель работы: освоить методы запуска больших языковых моделей *(LLM)* на *Android*-устройствах с использованием библиотеки *kotlinllamacpp* и *GGUF*-моделей (на примере *Qwen-2.5-1.5B-Instruct*).

Задание

Добавить в приложение по своему варианту генеративную *AI*-функцию (помечена красным цветом). Реализация должна быть выполнена по технологии *On-Device AI:* генерация и обработка данных должны происходить локально на устройстве, полностью в автономном режиме без обращения к внешним *API* и сети Интернет.

Технологический стек

– Библиотека: *llamacpp-kotlin*;
– Модель: *Qwen-2.5-1.5B-Instruct (GGUF Q4_K_M)*.

Реализация *AI*-функционала должна включать:

1. Интерфейс *LlmRepository* – абстракция над *LlamaHelper* из *kotlinllamacpp*.
2. Реализацию *LlmRepositoryImpl* – загрузка модели и генерация через *LlamaHelper*.
3. *UseCase* – формирование промпта, вызов *LLM*, постобработка результата.
4. Обработку состояния *Loading/Success/Error* – через *StateFlow*, включая отдельное состояние загрузки модели.
5. Кэширование результатов в БД.
6. Обрабатку офлайн-режима – модель работает локально, но должна быть проверка совместимости устройства и *graceful degradation* при недоступности.

Контрольные вопросы

1. Какие параметры генерации использовались в вашем приложении и как они повлияли на результат?
2. Какие элементы промпта использовались в вашей задаче и как они влияют на качество ответа?
3. Какие характеристики *Android*-устройства определяют, запустится ли выбранная *GGUF*-модель?
4. Как в *kotlinllamacpp* организована потоковая генерация через события *LLMEvent*?
5. В чём смысл кеширования результатов генерации при офлайн-работе модели?
6. Как размер контекстного окна влияет на длину промпта и генерируемого ответа?
7. Что такое *thermal throttling* и как он влияет на генерацию *LLM* на устройстве?
8. Как выбрать оптимальное значение *contextLength* при загрузке модели?
9. Как корректно освобождать ресурсы после завершения работы с *LLM*?

Лабораторная работа №6

Разработка голосового интерфейса в *Android*-приложениях:
распознавание и синтез речи

Цель работы: освоить методы интеграции речевых технологий в *Android*-приложения. Изучить принципы работы систем распознавания речи *(Speech-to-Text)* и синтеза речи *(Text-to-Speech)*. Овладеть навыками запроса разрешений на доступ к микрофону, управления жизненным циклом аудиосервисов и обработки ошибок распознавания.

Задание

Добавить в приложение по своему варианту голосовой интерфейс (помечен серым цветом). Реализация должна включать как голосовой ввод *(STT)*, так и озвучивание ответов *(TTS)*.

Технологический стек

- Распознавание речи: *SpeechRecognizer*;
- Синтез речи: *TextToSpeech*.

Реализация голосового функционала должна включать:

1. Интерфейс *VoiceRepository* – абстракция над *SpeechRecognizer* и *TextToSpeech*.
2. Реализацию *VoiceRepositoryImpl* – инициализация сервисов, запуск прослушивания, озвучивание текста.
3. Состояния VoiceUiState – обработка статусов *Idle, Listening, Processing, Error* через *StateFlow*.
4. Изоляцию от *UI* – *SpeechRecognizer* и *TextToSpeech* не должны вызываться напрямую из *Composable*-функций.
5. Управление жизненным циклом – обязательное освобождение ресурсов (*destroy() / shutdown())* в *ViewModel.onCleared()).*
6. Обработку ошибок – отсутствие разрешения *RECORD_AUDIO*, микрофон занят, языковая модель не поддерживается, ошибки распознавания.

Контрольные вопросы

1. Какие разрешения необходимы для работы *SpeechRecognizer* и как их корректно запросить в *Jetpack Compose*?
2. В чём разница между *SpeechRecognizer* и *RecognizerIntent.ACTION_RECOGNIZE_SPEECH*? Какой подход использовался в разработанном приложении?
3. Как работает *RecognitionListener*? Какие методы вызываются при успешном распознавании и при ошибке?
4. Почему *SpeechRecognizer* и *TextToSpeech* нельзя инициализировать внутри *Composable*-функции?

5. Как освободить ресурсы *TextToSpeech* и *SpeechRecognizer* при завершении работы *ViewModel*?
6. Что такое *EXTRA_PREFER_OFFLINE* и какие ограничения он накладывает на распознавание речи?
7. Какие проблемы могут возникнуть при работе *SpeechRecognizer* на устройствах без сервисов *Google*?
8. Как реализовать парсинг чисел из голосового ввода?
9. Как связать голосовой ввод с существующей архитектурой *MVVM* и *UDF*, чтобы не нарушить однонаправленный поток данных?
10. Какие ограничения есть у *TextToSpeech* при озвучивании длинных текстов и как их обойти?

Вариант 1

Мобильное приложение для учета домашней библиотеки с *ISBN* сканером.

Функциональная часть приложения

Приложение должно обеспечивать выполнение следующих ключевых функций:

1. **Каталогизация и просмотр (Главный экран):**
   - Визуализация коллекции книг в виде виртуальной полки;
   - Поиск по автору или названию;
   - Фильтрация по жанрам и статусам (*«Прочитано»*, *«Читаю»*).
2. **Просмотр детальной информации:**
   - Отображение карточки книги с обложкой, аннотациями, годом издания, *ISBN* и личной оценкой пользователя.
3. **Автоматизированное добавление:**
   - Сканирование штрихкода *ISBN* на обратной стороне книги через камеру устройства.
4. **Управление данными:**
   - Сетевой слой на базе *Retrofit* и *Kotlin Coroutines* отправляет идентифицированный *ISBN* в стороннее *REST API* (Например, *Open Library* - https://openlibrary.org/dev/docs/api/search). Полученный ответ автоматически обрабатывается и сущность сохраняется в базу данных;
   - Предусмотреть возможность ручного редактирования карточек.
5. ***AI*-функция:**
   - Генератор аннотаций – по названию, автору и жанру книги создается краткая аннотация в 2-3 предложения.

     Пример промпта: Ты – литературный редактор. Напиши краткую аннотацию для книги «Мастер и Маргарита» Михаила Булгакова в жанре мистика. Требования: 2-3 предложения, без спойлеров.
6. **Голосовой интерфейс:**
   - Пользователь голосом вводит название книги для поиска, а приложение озвучивает найденную аннотацию.

Архитектурные особенности приложения

В качестве централизованного механизма внедрения зависимостей для всех архитектурных слоев приложения должен быть использован фреймворк *Koin*.

Хранение данных

В работе должен быть реализован следующий подход к хранению информации:

1. **Основная база данных (Реляционная):**
   - Хранит каталог книг, сущности авторов, жанровые связи, и т.д.;
   - Взаимодействие с БД осуществляется с помощью библиотеки *Room*.
2. **Вспомогательное хранилище (*Jetpack DataStore*):**

– Используется для сохранения пользовательских конфигураций интерфейса (параметры сортировки списков, выбранный режим отображения сетки) и аналитических целей чтения (например, установленный лимит прочитанных книг на текущий год).

Вариант 2

Мобильное приложение для автоматизации квестов со встроенным модулем считывания контрольных меток.

Функциональная часть приложения

Приложение должно обеспечивать выполнение следующих ключевых функций:

1. **Каталогизация и просмотр (Главный экран):**
   – Отображение списка доступных игровых сценариев;
   – Отображение текущего прогресса, набранных баллов и журнала времени прохождения.
2. **Просмотр детальной информации:**
   – Экран текущего этапа игры: вывод текстовой загадки, подсказок и условий перехода на следующий уровень.
3. **Автоматизированное добавление:**
   – Сканирование физического *QR*-кода на игровой локации с помощью камеры устройства.
4. **Управление данными:**
   – Реактивный конвейер на базе *Kotlin Coroutines SharedFlow*. Компонент сканера публикует событие обнаружения метки в общую шину событий, которая асинхронно перенаправляет его в изолированный модуль валидации. Валидатор осуществляет верификацию хэш-суммы на соответствие заданному сценарию, обновляет состояние прогресса в локальной базе данных и инициирует смену состояния пользовательского интерфейса.
5. ***AI*-функция:**
   – Генератор подсказок – создаются текстовые подсказки к загадке, которые постепенно раскрывают ответ, не выдавая его сразу.
   Пример промпта: Ты – автор квестов. Придумай 3 подсказки к загадке: «Висит груша, нельзя скушать». Каждая следующая подсказка должна быть чуть более явной. Ответ: 3 пункта.
6. **Голосовой интерфейс:**
   – Пользователь голосом произносит свой ответ на загадку, а приложение распознаёт его, проверяет на соответствие сценарию и озвучивает вердикт (успех или предложение взять подсказку).

Архитектурные особенности приложения

В качестве централизованного механизма внедрения зависимостей для всех архитектурных слоев приложения должен быть использован фреймворк *Dagger Hilt*.

Хранение данных

В работе должен быть реализован следующий подход к хранению информации:

1. **Основная база данных (Реляционная):**
   - Хранит профиль игрока, историю прохождения и временные метки;
   - Взаимодействие с БД осуществляется с помощью встроенного фреймворка *Room*;

2. **Вспомогательное хранилище (Нереляционная БД, *Realm*):**
   - Содержит полный сценарий игры (уникальный *id* этапа, текст загадки, хэш правильного ответа, количество баллов).

Вариант 3

Мобильное приложение для учета показаний ЖКХ со встроенным модулем распознавания счетчиков.

Функциональная часть приложения

Приложение должно обеспечивать выполнение следующих ключевых функций:

1. **Каталогизация и просмотр (Главный экран):**
   - Отображение списка подключенных приборов учета (вода, газ, свет);
   - Фильтрация по типам ресурсов;
   - Интеграция графиков потребления через *Compose Charts*.
2. **Просмотр детальной информации:**
   - Хронологический список всех зафиксированных показаний по конкретному прибору;
   - Расчет разницы с прошлым месяцем;
   - Вывод итоговой стоимости на основе тарифов.
3. **Автоматизированное добавление:**
   - Сканирование числового табло физического счетчика через камеру с автоматическим извлечением цифр.
4. **Управление данными:**
   - Функция экспорта показателей по каждому прибору за указанный период времени.
5. ***AI*-функция:**
   - Прогноз показаний – предсказываются ожидаемые показания на следующий месяц на основе данных за прошлые месяцы и времени года.
     Пример промпта: Ты – аналитик ЖКХ. Спрогнозируй показания счётчика воды на июль на основе показаний за прошлые годы. Семья из 3 человек.

Летом ожидается жара +30°C. Укажи диапазон (минимум-максимум). Ответ: число + 2-3 предложения объяснения.

6. **Голосовой интерфейс:**
   - Пользователь голосом диктует показания счётчиков, а приложение озвучивает разницу с прошлым месяцем.

Архитектурные особенности приложения

В качестве централизованного механизма внедрения зависимостей для всех архитектурных слоев приложения должен быть использован фреймворк *Koin*.

Хранение данных

В работе должен быть реализован следующий подход к хранению информации:

1. **Основная база данных (Реляционная, *SQL Lite*):**
   - Изначально пустая и заполняется/обновляется начальными данными через *REST API* каждый раз при старте приложения. При этом, данные, добавленные пользователем во время предыдущих сессий, не должны теряться);
   - Взаимодействие с БД осуществляется с помощью библиотеки *Room*.
2. **Вспомогательное хранилище (Нереляционная БД, *Realm*):**
   - Содержит тарифную сетку по регионам и типам ресурсов (уникальный *id*, базовая стоимость за куб/кВт, нормативы).

Вариант 4

Мобильное приложение для финансового планирования с модулем импорта банковских выписок.

Функциональная часть приложения

Приложение должно обеспечивать выполнение следующих ключевых функций:

1. **Каталогизация и просмотр (Главный экран):**
   - Отображение баланса кошельков и списка транзакций;
   - Фильтрация по категориям затрат;
   - Визуализация структуры расходов через круговые диаграммы *Compose Charts*.
2. **Просмотр детальной информации:**
   - Отображение подробной карточки транзакции с указанием *MCC*-кода точки, даты, времени и привязанного счета.
3. **Автоматизированное добавление:**
   - Автоматический импорт массива операций через системный компонент выбора файлов (*.ActivityResultContracts.OpenDocument*).
4. **Управление данными:**
   - Реактивный конвейер на базе *Kotlin Coroutines Flow*, осуществляющий асинхронное чтение загруженного пользователем внешнего файла,

валидацию его структуры, автоматическое сопоставление *MCC*-кодов со встроенным *JSON*-справочником и пакетную (массовую) конвертацию данных в транзакционные записи локальной базы данных.

5. ***AI*-функция:**
   - Аналитика расходов – по списку транзакций за месяц генерируется связный финансовый отчёт с выводами: где выросли траты, какие категории доминируют, что можно оптимизировать.
     Пример промпта: Ты – персональный финансовый аналитик. Проанализируй траты за месяц и напиши краткий отчёт и дай 3 рекомендации: что выросло, на чём можно сэкономить.
   - Пользователь голосом диктует показания счётчиков, а приложение озвучивает разницу с прошлым месяцем.
6. **Голосовой интерфейс:**
   - Пользователь голосом диктует сумму и категорию траты, а приложение озвучивает остаток бюджета.

Архитектурные особенности приложения

В качестве централизованного механизма внедрения зависимостей для всех архитектурных слоев приложения должен быть использован фреймворк *Koin*.

Хранение данных

В работе должен быть реализован следующий подход к хранению информации:

1. **Основная база данных (Реляционная):**
   - Взаимодействие с БД осуществляется с помощью библиотеки *Room*.
2. **Вспомогательное хранилище (Статический *JSON*-файл):**
   - Локальный файл *JSON,* размещенный в директории *assets*, содержащий международный справочник *MCC*-кодов эквайринга (уникальный *id*, наименование категории, иконка).

Вариант 5

Мобильное приложение для инвентаризации имущества со встроенным модулем генерации *QR*-кодов.

Функциональная часть приложения

Приложение должно обеспечивать выполнение следующих ключевых функций:

1. **Каталогизация и просмотр (Главный экран):**
   - Иерархическое отображение структуры хранения (Комната → Контейнер → Вещь);
   - Текстовый поиск по вещам с указанием их точного местоположения.
2. **Просмотр детальной информации:**
   - Отображение карточки контейнера с перечнем всех находящихся внутри материальных ценностей;

- Вывод программно-сгенерированного *QR*-кода маркировки на экран.

3. **Автоматизированный поиск:**
   - Сканирование *QR*-кода на коробке через камеру устройства и вывод списка вещей.
4. **Управление данными:**
   - Функция экспорта списка вещей конкретной коробки в виде отчуждаемого файла данных для обмена с другими пользователями;
   - Полный цикл *CRUD*-операций для вещей и контейнеров.
5. ***AI*-функция:**
   - Оптимизация хранения – предлагается оптимальный порядок хранения с учётом хрупкости, веса, габаритов.
     Пример промпта: Ты – эксперт по организации хранения. Проанализируй содержимое шкафа и предложи оптимальную раскладку с учётом веса, хрупкости и габаритов. Объясни, почему каждая вещь должна быть именно там. Ответ: структурированный список по полкам с обоснованием.
6. **Голосовой интерфейс:**
   - Пользователь голосом отдаёт команду на перемещение вещи, а приложение голосом подтверждает её новое местоположение.

Архитектурные особенности приложения

В качестве централизованного механизма внедрения зависимостей для всех архитектурных слоев приложения должен быть использован фреймворк *Dagger Hilt*.

Хранение данных

В работе должен быть реализован следующий подход к хранению информации:

1. **Основная база данных (Реляционная):**
   - Изначально пустая и заполняется/обновляется начальными данными через *REST API* каждый раз при старте приложения. При этом, данные, добавленные пользователем во время предыдущих сессий, не должны теряться);
   - Взаимодействие с БД осуществляется с помощью библиотеки Room.
2. **Вспомогательное хранилище (Статический *JSON*-файл):**
   - Локальный файл *JSON*, содержащий классификатор типов имущества (уникальный *id*, категория, иконка);
   - Приложение должно считывать созданный *JSON*-файл при запуске и использовать его для классификации типов имущества.

Вариант 6

Мобильное приложение для каталогизации дисконтных карт со встроенным модулем сканирования штрихкодов.

Функциональная часть приложения

Приложение должно обеспечивать выполнение следующих ключевых функций:

1. **Каталогизация и просмотр (Главный экран):**
   - Отображение списка всех добавленных карт;
   - Быстрый поиск по названию карты или магазина;
   - Группировка и фильтрация карт по встроенным категориям (например: *«Супермаркеты», «Одежда», «Кафе и рестораны»*).
2. **Просмотр детальной информации:**
   - Отображение названия магазина, штрихкода (в виде графического изображения) и числового номера под ним;
   - Возможность присвоения карте индивидуального цветового оформления для визуального отличия в общем списке.
3. **Автоматизированное добавление:**
   - Сканирование физической карты с помощью камеры устройства.
4. **Управление данными:**
   - Возможность ручного редактирования информации о карте (изменение названия, категории, цвета) и функция удаления карты из базы данных.
5. ***AI*-функция:**
   - Смысловая ревизия – анализируется вся коллекция карт, находится те, что лежат не в своих категориях, и предлагается пересортировка с обоснованием.

     Пример промпта: Ты – ассистент по организации коллекций. Проверь, правильно ли распределены карты по категориям. Найди карты, которые лежат не в своих категориях, и предложи правильные. Объясни, почему каждая карта должна быть перемещена. Ответ: список карт с обоснованием.
6. **Голосовой интерфейс:**
   - Пользователь голосом называет магазин, а приложение озвучивает категорию его дисконтной карты.

Архитектурные особенности приложения

В качестве централизованного механизма внедрения зависимостей для всех архитектурных слоев приложения должен быть использован фреймворк *Koin*.

Хранение данных

В работе должен быть реализован следующий подход к хранению информации:

1. **Основная база данных (Реляционная):**
   - Изначально пустая. Заполняется/обновляется начальными данными через *REST API* каждый раз при старте приложения. При этом, данные,

добавленные пользователем во время предыдущих сессий, не должны теряться);
- Взаимодействие с БД осуществляется с помощью библиотеки *Room*.

2. **Вспомогательное хранилище (Статический *JSON*-файл):**
   - Файл, размещенный в директории *assets*, содержит массив системных категорий (уникальный *id*, наименование, иконка);
   - Приложение должно считывать созданный *JSON*-файл при запуске, обрабатывать его через механизм десериализации и использовать для классификации карт. Вносить изменения в этот файл из приложения нельзя.

Вариант 7

Мобильный органайзер домашних растений.

Функциональная часть приложения

Приложение должно обеспечивать выполнение следующих ключевых функций:

1. **Каталогизация и просмотр (Главный экран):**
   - Отображение списка комнатных растений;
   - Интерактивный календарь процедур;
   - Визуальные индикаторы необходимости полива (смена цвета карточки при просрочке).
2. **Просмотр детальной информации:**
   - Журнал ухода за растением (история поливов, подкормок, пересадок);
   - Вывод ботанической справки по условиям содержания (свет, влажность).
3. **Автоматизированное добавление:**
   - Сканирование текстового наименования цветка с этикетки или ценника через камеру устройства.
4. **Управление данными:**
   - Отсканированное наименование отправляется асинхронным сетевым запросом через *REST API* энциклопедии растений (например, https://perenual.com/docs/api). Приложение скачивает базовый регламент ухода, сохраняет его в базу данных и автоматически генерирует цепочку локальных напоминаний через службу *WorkManager*.
5. ***AI*-функция:**
   - Диагностика растения – пользователь описывает симптомы (желтеют листья, сохнет земля, пятна), а приложение с помощью *LLM* предполагает причину и даёт совет по уходу.

     Пример промпта: Ты – опытный садовод. У замиокулькаса ержелтеют нижние листья, земля сухая. Что может быть причиной и что делать? Ответ: причина (1 предложение) + рекомендация (1-2 предложения).
6. **Голосовой интерфейс:**
   - Пользователь голосом фиксирует выполненную процедуру ухода, а приложение озвучивает дату следующего полива.

Архитектурные особенности приложения

В качестве централизованного механизма внедрения зависимостей для всех архитектурных слоев приложения должен быть использован фреймворк *Dagger Hilt*.

Хранение данных

В работе должен быть реализован следующий подход к хранению информации:

1. **Основная база данных (Реляционная):**
   - Хранит карточки растений, логи процедур, расписание триггеров и т.д.;
   - Взаимодействие с БД осуществляется с помощью библиотеки *Room*.
2. **Вспомогательное хранилище (Нереляционная БД, *Realm*):**
   - Содержит базовый справочник по семействам растений и типам удобрений (уникальный *id*, название, иконка).

Вариант 8

Мобильное приложение для планирования цифрового гардероба и сборки чемоданов в поездки с функцией расчета веса багажа.

Функциональная часть приложения

Приложение должно обеспечивать выполнение следующих ключевых функций:

1. **Каталогизация и просмотр (Главный экран):**
   - Иерархическое отображение структуры хранения вещей (Сезон/Стиль → Образ → Конкретная вещь);
   - Текстовый поиск по вещам с фильтрацией по статусу («В шкафу», «В поездке», «В стирке»).
2. **Просмотр детальной информации:**
   - Отображение карточки поездки (чемодана) с перечнем всех упакованных вещей и автоматическим расчетом их суммарного веса;
   - Вывод прогресс-бара, показывающего загрузку чемодана относительно лимитов авиакомпании (например, до 10 кг или до 23 кг).
3. **Автоматизированный сбор:**
   - Автоматическое формирование базового списка вещей на основе типа поездки (например, «Пляж», «Бизнес», «Горы») и погодных условий в точке назначения.
4. **Управление данными:**
   - Функция экспорта финального чек-листа собранного чемодана в виде отчуждаемого файла данных для обмена с попутчиками.
5. ***AI*-функция:**
   - Генератор образов – по типу поездки и погоде предлагается список вещей и стилевые сочетания.

     Пример промпта: Ты – стилист. Поездка на море на 5 дней, температура +28. Собери список одежды: 3 верха, 2 низа, обувь, аксессуары. Ответ: структурированный список.

6. **Голосовой интерфейс:**
    - Пользователь голосом перечисляет вещи для поездки, а приложение озвучивает их общий вес.

Архитектурные особенности приложения

В качестве централизованного механизма внедрения зависимостей для всех архитектурных слоев приложения должен быть использован фреймворк *Koin*.

Хранение данных

В работе должен быть реализован следующий подход к хранению информации:

1. **Основная база данных (Реляционная):**
    - Изначально пустая и заполняется/обновляется начальными данными через *REST API* каждый раз при старте приложения. При этом, данные, добавленные пользователем во время предыдущих сессий (новые добавленные вещи, созданные поездки), не должны теряться;
    - Взаимодействие с БД осуществляется с помощью библиотеки *Room*.
2. **Вспомогательное хранилище (*Jetpack DataStore*):**
    - Хранит глобальные настройки приложения (лимит веса багажа по умолчанию, выбранная тема оформления (системная, темная, светлая), а также маркер первого запуска приложения). Данные настройки можно изменять в специальном меню внутри приложения.

Вариант 9

Мобильное приложение для каталогизации настольных игр и учета игровых сессий.

Функциональная часть приложения:

Приложение должно обеспечивать выполнение следующих ключевых функций:

1. **Каталогизация и просмотр (Главный экран):**
    - Отображать коллекцию настольных игр владельца;
    - Осуществлять быстрый поиск по количеству игроков или времени партии;
    - Хранить и отображать истории сыгранных матчей.
2. **Просмотр детальной информации:**
    - Карточка игры (минимальный/максимальный возраст, рейтинг, сложность);
    - Интерактивный бланк для цифрового подсчета победных очков в конце партии.
3. **Автоматизированное добавление:**
    - Сканирование штрихкода на коробке с настольной игрой с помощью камеры устройства.
4. **Управление данными:**
    - При инициализации сессии посредством компонента *SharedFlow (Kotlin Coroutines)* осуществляется асинхронная рассылка события *MatchStarted*. После этого, запускается независимый таймер раундов. База данных

блокирует статус игры как *«активна»*, а интерфейс переключается в режим скоринга. Результаты матча сериализуются и сохраняются в историю.

5. ***AI*-функция:**
   - Разбор правил – по названию игры даётся краткое объяснение правил для новичка.
     Пример промпта: Ты – опытный настольщик. Объясни правила игры «Каркассон» для новичка. Кратко, 5–7 предложений, только базовые правила, без спойлеров стратегии.
6. **Голосовой интерфейс:**
   - Пользователь голосом вводит имя победителя и его очки, а приложение озвучивает итоговый счёт партии.

Архитектурные особенности приложения

В качестве централизованного механизма внедрения зависимостей для всех архитектурных слоев приложения должен быть использован фреймворк *Dagger Hilt*.

Хранение данных

В работе должен быть реализован следующий подход к хранению информации:

1. **Основная база данных (Реляционная):**
   - Хранит реестр коробок, профили участников/друзей, история сыгранных партий, статистика побед;
   - Взаимодействие с БД осуществляется с помощью библиотеки Room.
2. **Вспомогательное хранилище (Статический *JSON*-файл):**
   - Локальный файл *JSON (Decodable)*, содержащий математические шаблоны и правила начисления очков для различных типов настольных игр (уникальный *id* игры, перечень критериев подсчета, лимиты).

Вариант 10

Мобильное приложение для проверки состава продуктов со сканером штрихкодов.

Функциональная часть приложения

Приложение должно обеспечивать выполнение следующих ключевых функций:

1. **Каталогизация и просмотр (Главный экран):**
   - Отображение истории проверенных товаров;
   - Персональный профиль пользователя с указанием критических пищевых аллергенов;
   - Быстрый поиск по истории.
2. **Просмотр детальной информации:**
   - Полный разбор состава продукта: выделение опасных компонентов цветом, расшифровка *Е*-добавок, вывод вердикта (*«Безопасно»* или *«Противопоказано»*).

3. **Автоматизированное добавление:**
   - Сканирование штрихкода на упаковке товара с помощью камеры устройства.
4. **Управление данными:**
   - Асинхронный сетевой *REST*-запрос к базе данных *Open Food Facts* (https://world.openfoodfacts.org/data) или аналогам.
   - Реализовать логику дублирования (*Fallback*): если штрихкод отсутствует в сети, приложение предлагает переключить камеру в текстовый режим для прямого сканирования блока «Состав» с упаковки и сопоставления слов с медицинским реестром.
5. ***AI*-функция:**
   - Анализ рациона – пользователь вводит список съеденных продуктов за день, а приложение выдаёт полный разбор рациона: калорийность, белки, жиры, углеводы, а также выявляет скрытые аллергены из профиля пользователя. По итогу даёт общую оценку – насколько рацион сбалансирован и что стоит изменить.

     Пример промпта: Ты – диетолог. Проанализируй рацион за день. Норма: 2000 ккал, БЖУ 90/70/250г. Аллергии: лактоза, арахис. Съедено: овсянка с молоком, кофе со сливками, паста с сыром, салат с ореховым соусом, печенье с шоколадом, пицца с колбасой и сыром. Посчитай КБЖУ, сравни с нормой, найди скрытые аллергены, дай 2-3 рекомендации. Ответ: таблица БЖУ + список аллергенов + вывод.
6. **Голосовой интерфейс:**
   - Пользователь голосом перечисляет съеденные продукты, а приложение озвучивает предупреждение об обнаруженных аллергенах.

Архитектурные особенности приложения

В качестве централизованного механизма внедрения зависимостей для всех архитектурных слоев приложения должен быть использован фреймворк *Koin*.

Хранение данных

В работе должен быть реализован следующий подход к хранению информации:

1. **Основная база данных (Реляционная):**
   - Хранит профиль аллергий пользователя, журнал проверенных продуктов т.д.;
   - Взаимодействие с БД осуществляется с помощью библиотеки *Room*.
2. **Вспомогательное хранилище (Нереляционная БД, *Realm*):**
   - Содержит медицинский реестр вредных химических добавок и аллергенов (уникальный *id*, общепринятый маркер, описание воздействия, степень риска).

Вариант 11

Мобильный помощник автолюбителя со встроенным модулем распознавания *VIN*-номеров.

Функциональная часть приложения

Приложение должно обеспечивать выполнение следующих ключевых функций:

1. **Каталогизация и просмотр (Главный экран):**
   - Отображение профиля транспортного средства, текущего пробега, списка предстоящих сервисных работ, графиков затрат на содержание автомобиля.
2. **Просмотр детальной информации:**
   - Отображение журнала технического обслуживания (история замены масел, фильтров, деталей) с привязкой к датам и пробегу.
3. **Автоматизированное добавление:**
   - Оптический захват *VIN*-номера автомобиля с документов или кузова с помощью камеры устройства.
4. **Управление данными:**
   - Реализовывать сетевой модуль на базе *Retrofit* и *Kotlin Coroutines*, который отправляет *VIN*-номер в *REST API* (например, https://vpic.nhtsa.dot.gov/api/) спецификаций автомобилей. Полученные технические параметры (модель, двигатель, год) сопоставляются с технологическими картами регламентов ТО, автоматически генерируя индивидуальный план замен в базе данных.
5. ***AI*-функция:**
   - Автодиагностика – по описанию симптомов (стук, вибрация, запах, поведение машины) предполагаются возможные причины неисправностей и рекомендует, что проверить в первую очередь, с учётом модели, пробега и истории обслуживания.

     Пример промпта: Ты – автомеханик. Машина *Toyota Camry* 2020, пробег 80 000 км. При торможении на 60-80 км/ч вибрация в руле, педаль пульсирует. Назови 2-3 возможные причины и что проверить в первую очередь. Ответ: список причин с приоритетом.
6. **Голосовой интерфейс:**
   - Пользователь голосом описывает посторонние звуки в машине, а приложение озвучивает возможные причины неисправности.

Архитектурные особенности приложения

В качестве централизованного механизма внедрения зависимостей для всех архитектурных слоев приложения должен быть использован фреймворк *Koin*.

Хранение данных

В работе должен быть реализован следующий подход к хранению информации:

1. **Основная база данных (Реляционная):**
   - Хранит карточки автомобилей, историю ремонтов, счетчики пробега т.д.;

- Взаимодействие с БД осуществляется с помощью библиотеки *Room*.

2. **Вспомогательное хранилище (Нереляционная БД, *Realm*):**
   - Содержит заводские технологические карты обслуживания (уникальный *id* регламента, интервал замены в километрах, перечень расходников).

Вариант 12

Мобильное приложение для оцифровки визитных карт.

Функциональная часть приложения

Приложение должно обеспечивать выполнение следующих ключевых функций:

1. **Каталогизация и просмотр (Главный экран):**
   - Отображение списка контактов;
   - Группировку по компаниям и тегам;
   - Поиск по должностям и/или фамилии.
2. **Просмотр детальной информации:**
   - Отображение цифровой карточки контакта с кнопками (*«Позвонить»*, *«Написать email»*);
   - Просмотр истории встреч и заметок.
3. **Автоматизированное добавление:**
   - Сканирование бумажной визитки камерой устройства.
4. **Управление данными:**
   - Реактивный конвейер на базе *Kotlin Coroutines Flow*. Модуль сканирования публикует событие *ContactParsed*, после чего поток данных асинхронно маршрутизирует его к независимым потребителям: первый модуль осуществляет валидацию данных, второй производит пакетную запись в локальную базу данных, а третий – инициирует экспорт контакта в системную телефонную книгу посредством компонента *ContactsContract*.
5. ***AI*-функция:**
   - Итоги встречи – по заметкам о встрече генерируется краткое резюме (2-3 предложения) и список задач к выполнению с учётом информации о контакте.

     Пример промпта: Ты – бизнес-ассистент. Составь итоги встречи. Контакт: Иван Петров, «ТехноСтарт», директор по развитию. Заметки: обсуждали бюджет на квартал, поставка – конец марта, отправить КП до пятницы, включить вариант с рассрочкой. Сделай резюме (2-3 предложения) и 3 *follow-up* задачи.
6. **Голосовой интерфейс:**
   - Пользователь голосом надиктовывает заметку о встрече, а приложение озвучивает список задач на завтра.

Архитектурные особенности приложения

В качестве централизованного механизма внедрения зависимостей для всех архитектурных слоев приложения должен быть использован фреймворк *Dagger Hilt*.

Хранение данных

В работе должен быть реализован следующий подход к хранению информации:

1. **Основная база данных (Реляционная):**
   - Хранит метаданные контактов, историю взаимодействий, теги и т.д.;
   - Взаимодействие с БД осуществляется с помощью библиотеки *Room*.
2. **Вспомогательное хранилище (Нереляционная БД, *Realm*):**
   - Содержит справочник телефонных кодов стран и масок ввода номеров для автоформатирования.

Вариант 13

Мобильное приложение для контроля домашней аптечки и сроков годности медикаментов.

Функциональная часть приложения

Приложение должно обеспечивать выполнение следующих ключевых функций:

1. **Каталогизация и просмотр (Главный экран):**
   - Отображение списка лекарств в наличии;
   - Сортировку по критическому сроку годности;
   - Фильтрацию по форме выпуска (таблетки, сиропы).
2. **Просмотр детальной информации:**
   - Карточка препарата с указанием остатка, дозировки, правил приема и текстовой инструкции.
3. **Автоматизированное добавление:**
   - Сканирование текстового коммерческого названия лекарства с упаковки с помощью камеры устройства.
4. **Управление данными:**
   - При добавлении препарата его название посредством реактивных потоков данных *Kotlin Flow* сопоставляется с локальной *JSON*-матрицей опасных лекарственных взаимодействий. В случае обнаружения конфликта с текущим курсом лечения система инициирует критическое предупреждение и блокирует транзакцию. При успешной валидации регистрирует график приемов в системном сервисе *AlarmManager*.
5. ***AI*-функция:**
   - Безопасная замена – когда препарат конфликтует с текущим курсом лечения или просрочен, приложение с помощью *LLM* предлагает 2-3 альтернативы из той же фармакологической группы, которые не конфликтуют с принимаемыми лекарствами, и объясняет, чем они

отличаются: действующим веществом, формой выпуска, особенностями приёма.

Пример промпта: Ты – фармаколог. Пациент принимает аспирин. Врач назначил ибупрофен, но матрица показывает конфликт: «НПВС + антиагрегант – риск ЖКК». Предложи 2-3 альтернативы ибупрофену из группы обезболивающих, не конфликтующие с аспирином. Для каждой: действующее вещество, отличие от ибупрофена, как принимать. Ответ: список из 2-3 пунктов.

6. **Голосовой интерфейс:**
   - Пользователь голосом называет лекарство, а приложение озвучивает вердикт о его совместимости с текущим курсом лечения.

Архитектурные особенности приложения

В качестве централизованного механизма внедрения зависимостей для всех архитектурных слоев приложения должен быть использован фреймворк *Koin*.

Хранение данных

В работе должен быть реализован следующий подход к хранению информации:

1. **Основная база данных (Реляционная):**
   - Хранит учет упаковок, даты производства, остатки, журнал приемов.
   - Взаимодействие с БД осуществляется с помощью библиотеки Room.
2. **Вспомогательное хранилище (Статический *JSON*-файл):**
   - Локальный файл *JSON (Decodable)*, содержащий матрицу несовместимости действующих веществ медицинских препаратов (уникальный *id*, вещество А, вещество Б, описание угрозы).

Вариант 14

Мобильный медицинский дневник со сканером бланков лабораторных анализов.

Функциональная часть приложения

Приложение должно обеспечивать выполнение следующих ключевых функций:

1. **Каталогизация и просмотр (Главный экран):**
   - Отображение хронологической ленты состояния здоровья;
   - Построение графиков изменения физиологических показателей;
   - Интерактивный (с помощью готовых графических элементов) ввод симптомов.
2. **Просмотр детальной информации:**
   - Отображение карточки конкретного лабораторного исследования (например, общий анализ крови) с наглядным выделением маркеров, вышедших за границы нормы.
3. **Автоматизированное добавление:**

- Сканирование печатного листа с результатами анализов из лаборатории через камеру смартфона.

4. **Управление данными:**
   - Сканер распознает текстовую таблицу и осуществляется сопоставление данных: выделенные показатели сверяются со встроенным справочником медицинских норм (хранится в *Realm*). Посредством компонента *SharedFlow (Kotlin Coroutines)* организуется асинхронная цепочка обработки: распознанные показатели реактивно объединяются с данными физической активности, импортируемыми из системного сервиса *Health Connect*.

5. ***AI*-функция:**
   - Разбор анализа – когда показатель выходит за пределы нормы, *LLM* объясняет простыми словами, что это значит: возможные причины, насколько это серьёзно, что делать.

     Пример промпта: Ты – терапевт. Объясни пациенту результат анализа простыми словами. Гемоглобин – 110 г/л, норма 120-160. Что это значит? Насколько серьёзно? Причины? Что делать? Ответ: 3-4 предложения.
6. **Голосовой интерфейс:**
   - Пользователь голосом описывает свои симптомы, а приложение озвучивает общие рекомендации.

Архитектурные особенности приложения

В качестве централизованного механизма внедрения зависимостей для всех архитектурных слоев приложения должен быть использован фреймворк *Dagger Hilt*.

Хранение данных

В работе должен быть реализован следующий подход к хранению информации:

1. **Основная база данных (Реляционная):**
   - Хранит дневник симптомов, оцифрованные маркеры анализов и т.д.
   - Взаимодействие с БД осуществляется с помощью библиотеки *Room*.
2. **Вспомогательное хранилище (Нереляционная БД, *Realm*):**
   - Содержит реестр (например, https://helix.ru/kb/item/02-029) медицинских референсных значений (уникальный *id* показателя, норма для мужчин/женщин по возрастным группам).

Вариант 15

Мобильный офлайн-тренажер иностранных слов методом интервальных повторений.

Функциональная часть приложения:

Приложение должно обеспечивать выполнение следующих ключевых функций:

1. **Каталогизация и просмотр (Главный экран):**
   - Отображение списка личных словарей пользователя;
   - Отображение статистики изученных слов за неделю через *Compose Charts*;
   - Запуск сессии повторений.
2. **Просмотр детальной информации:**
   - Визуальная двусторонняя флэш-карточка слова (слово, транскрипция, перевод, пример использования, уровень запоминания).
3. **Автоматизированное добавление:**
   - Оптический захват незнакомого иностранного слова с помощью камеры устройства.
4. **Управление данными:**
   - Распознанное слово передается в реактивный поток данных *Kotlin Channels* (или *StateFlow*). Если слово присутствует во встроенном словаре, карточка заполняется автоматически.
   - Алгоритм интервальных повторений *(SuperMemo SM-2)* вычисляет точную дату следующего показа слова и модифицирует очередь в базе данных. На основе обновленной выборки система агрегирует целевые записи, аннулирует неактуальные задачи в системном планировщике *AlarmManager* и регистрирует новый триггер в службе локальных *Push*-уведомлений на расчетное время повторения.
5. ***AI*-функция:**
   - Генератор примеров – пользователь вводит слово (или сканирует его), а приложение с помощью *LLM* создаёт 3 новых примера предложений на изучаемом языке с переводом. Примеры адаптированы под уровень пользователя: для начинающих – простые конструкции, для продвинутых – сложные.

     Пример промпта: Ты – преподаватель английского. Слово *ephemeral*. Уровень: *Intermediate (B1)*. Придумай 3 примера предложений на английском с переводом и кратким пояснением. Разной сложности: простое, среднее, сложное.
6. **Голосовой интерфейс:**
   - Пользователь голосом произносит иностранное слово для проверки, а приложение озвучивает правильный перевод и пример использования.

Архитектурные особенности приложения

В качестве централизованного механизма внедрения зависимостей для всех архитектурных слоев приложения должен быть использован фреймворк *Koin*.

Хранение данных

В работе должен быть реализован следующий подход к хранению информации:

1. **Основная база данных (Реляционная):**
   - Хранит персональный словарь пользователя, коэффициенты забывания, даты повторов.
   - Взаимодействие с БД осуществляется с помощью библиотеки *Room*.
2. **Вспомогательное хранилище (Нереляционная БД, *Realm*):**
   - Содержит базовый словарь объёмом 1000 наиболее популярных иностранных слов с готовыми переводами и примерами для офлайн-работы.